

Reporte

Montiel Ramírez Brayan

28 de septiembre de 2026

Capítulo 1

Resumen

El proyecto cuenta con un servidor que acepta la conexión con múltiples clientes y les envía mensajes, completo de acuerdo a los requerimientos derivados del protocolo. A su vez, cuenta con un cliente incompleto según las especificaciones, ya que sólo puede conectarse a un servidor, registrarse, obtener la lista de usuarios, ver los cambios de estados, cambiar de estado y enviar y recibir mensajes en la sala pública; no cuenta con la funcionalidad de las habitaciones, desde crearlas hasta enviar mensajes a ellas.

Capítulo 2

Introducción

El proyecto consiste de dos programas ejecutables: un **cliente** y un **servidor**, para un chat que permita la comunicación entre múltiples usuarios al mismo tiempo, mediante los siguientes recursos:

1. **Sala principal:** Es el espacio donde todos los usuarios envían mensajes públicos para todos los que estén conectados al mismo servidor.
2. **Salas:** Son espacios destinados al envío y recepción de mensajes, creados por una persona, los cuales permiten el ingreso de miembros mediante invitaciones enviadas por los miembros de las mismas.
3. **Chats privados:** Espacios para la comunicación privada entre dos usuarios del chat.
4. **Estado del usuario:** Le indican a un usuario si otro se encuentra conectado al chat, ocupado o lejos de la computadora.

Según un conjunto de reglas establecidas para la comunicación entre el programa del cliente y el del servidor (llamado **protocolo**) el programa del cliente debe cumplir con las siguientes funciones:

1. Conectarse a un servidor.
2. Enviar mensajes y recibir mensajes del servidor.
3. Identificarse con un nombre único.
4. Solicitar la lista de usuarios de un chat y de una sala junto a sus estados.
5. Cambiar su estado entre:
 - Active/Activo: Conectado al chat.

- Away/Lejos: Lejos de la computadora.
 - Bussy/Ocupado.
6. Crear una nueva sala con un nombre único en el servidor.
 7. Invitar a un usuario a una sala de la que sea miembro.
 8. Unirse a una sala donde haya sido invitado.
 9. Enviar mensajes a un usuario, a una sala o al chat público.
 10. Desconectarse del chat.

Mientras que el servidor debe poder recibir, responder y manejar notificaciones a otros clientes de tal forma que todas estas funciones se cumplan, lo cual hace mediante la transmisión y recepción de datos vía un protocolo llamado TCP/IP, para más de un cliente a la vez, y usando el estándar JSON para los mensajes que envía y recibe.

Capítulo 3

Análisis y diseño

3.1. Computabilidad

El problema de crear un chat que utilice el protocolo indicado es computable, ya que consiste en la creación de dos programas (cliente y servidor) que utilizan algoritmos con un número finito de pasos, los cuales son las 12 funciones que describe el protocolo, que a su vez se descomponen en algoritmos con un número finito de pasos también.

3.2. Comunicación

La comunicación entre el cliente y el servidor se realiza mediante el uso de *sockets* y el protocolo TCP/IP, donde el servidor conecta su socket a su dirección IP local en algún puerto y comienza a escuchar y, posteriormente, múltiples clientes conectan sus sockets a la IP y puerto del servidor, lo cual establece un canal de comunicación entre ambos. Una vez conectados, la comunicación comienza con el usuario, haciendo uso de la interfaz. para mandar un mensaje al servidor desde su programa, pasando por los sockets, llegando al servidor, decodificando e interpretando el mensaje, para posteriormente ejecutar acciones que cambien los registros de datos dentro del servidor y responder al mensaje enviado o notificar al resto de clientes conectados sobre la modificación.

Por el lado del cliente, hay dos flujos de entrada y salida: uno de envío y otro de recepción de mensajes. A continuación, un ejemplo de ambos:

1. Flujo de envío: Si el programa del cliente ya está en comunicación con el servidor, el usuario ingresa su nombre mediante una interfaz (entrada),

esta valida que el formato sea correcto y luego se le notifica al cliente que envíe un mensaje que al servidor para identificarse. El flujo termina cuando cuando el mensaje es enviado (salida).

2. Flujo de recepción: El cliente se encuentra esperando una respuesta del servidor y recibe una que indica que la identificación del usuario fue exitosa (entrada), entonces notifica a la interfaz para que le muestre el chat (salida).

Por el lado del servidor hay un único flujo de entrada y múltiples salidas. A continuación un ejemplo:

1. El servidor se encuentra esperando recibir un mensaje y en cuanto lo recibe valida que tenga el formato correcto y lo interpreta (entrada). Este mensaje es una solicitud de un cliente para identificar a un usuario, entonces valida que el formato del nombre sea correcto o si ya existe en los registros y si no, entonces lo registra y le envía un mensaje de respuesta diciendo que la operación fue exitosa (salida), además de notificar al resto de usuarios ya conectados que hay un nuevo usuario en el chat (múltiples salidas).

La unión de estos flujos (los dos del cliente y el del servidor) da como resultado mínimo una entrada desde el usuario y una salida (posiblemente vacía) al mismo usuario, así que es suficiente para manejar al menos a un cliente; pero además, al esperar entradas y tener múltiples salidas (el servidor) se completa y resuelve el problema de mantener múltiples usuarios, dando como resultado la solución al problema de crear el chat, y por finalizado el análisis. Las posteriores decisiones se fundamentan en este análisis.

Más adelante se expone el motivo del paradigma escogido; sin embargo, a partir de ahora el diseño del cliente y servidor se inclinan a la orientación a objetos. Así mismo, se indica al lector que en el repositorio público de este proyecto se encuentran los diagramas necesarios para el diseño e implementación, así que aquí solamente se expondrá la parte teórica.

3.3. Diseño del servidor

El servidor funciona mediante las siguientes entidades y eventos:

- **Entidades que contienen y validan datos:** Son las entidades que almacenan datos (como nombres de usuario, estados, nombres de sala, miembros, invitaciones, etc.) y que validan que su estado sea el adecuado. Entre estas entidades se encuentran un *usuario*, una *sala* y una

entidad que valida que el formato de textos enviados a salas y usuarios sea el adecuado.

- **Encargados de la conexión:** Son entidades que se encargan de la comunicación directa con los sockets de los clientes para la escritura o lectura de mensajes.
- **Notificaciones de entrada/salida de mensajes:** para la entrada y la salida de mensajes del servidor.
- **Contenedor global de datos:** Contiene la información concreta de usuarios y salas existentes en el servidor y permite operar con ella.
- **Actores en el servidor:** Modifican el estado interno del servidor (sus datos, el estado de sus entidades, etc.) como respuesta a un mensaje recibido; además, tienen salida a otros clientes (manejan notificaciones).
- **Traductores de mensajes:** Traducen los mensajes recibidos y emitidos por el servidor a un formato apto para la transmisión de datos de sockets o a actores dentro del servidor.

Estas entidades se dividen, a su vez, en varios objetos implementados en el servidor y algunos mostrados en los diagramas de clases.

3.4. Diseño del cliente

El cliente funciona por medio de las siguientes entidades y eventos:

- **Entidades observadas:** Son entidades que almacenan datos (nombres de usuario, estados, etc.) y los validan, pero cuyas modificaciones (cambio de estado) pueden ser notadas por las entidades que las utilicen.
- **Conexiones con el servidor:** Son entidades encargadas de realizar la escritura y lectura directa de mensajes con el servidor.
- **Emisores y receptores de mensajes:** Sirven como puente o interfaz entre las acciones que puede realizar el usuario (descritas en el protocolo) y la emisión de dicho mensaje o la recepción de alguno, mediante las conexiones con el servidor.
- **Traductores de mensajes:** Sirven para traducir mensajes recibidos y emitidos por el cliente al servidor en un formato compatible con los sockets.

- **Proveedores de contexto:** Son todas las entidades que proveen datos y comandos como puente entre el usuario y la emisión/recepción de mensajes.
- **Vistas:** Son las entidades encargadas de comunicarse con los proveedores de contexto para comunicarse a su vez con el servidor. Dependen del contexto para saber qué mostrarle y permitirle hacer al usuario.

Capítulo 4

Implementación

4.1. Paradigma y Lenguaje

El paradigma orientado a objetos es el más adecuado para este problema, pues los usuarios no son más que contenedores de datos como nombres y estados, las doce funciones que describe el protocolo: las responsabilidades de uno o varios objetos, y el almacenamiento de datos: el uso de estructuras de datos dentro de algún objeto que permita la manipulación y acceso protegido a estos. Esto aplica para ambos programas.

El lenguaje a utilizar será **C#** junto con los frameworks de **.NET** y **Avalonia**, por los siguientes motivos:

- **C#** y **.Net**: Ambos programas cliente-servidor esperan mensajes, lo cual puede ser fácilmente abordado mediante el patrón **await** que este lenguaje y framework permiten, además de permitir la programación asíncrona, lo cual sirve necesario para manejar a múltiples clientes a la vez y para no bloquear la interfaz de ningún usuario.
- **Avalonia**: Ambos flujos del cliente descritos durante el análisis son implementables mediante una arquitectura MVVM, siendo el modelo el encargado de la comunicación con el servidor y definición de entidades (usuario, sala, etc.), la vista la interfaz del usuario y la vista-modelo quien los conecta, pero sin conocer a la vista y siendo un simple contenedor de contexto. Dado que los cambios dentro de las entidades de los modelos son visibles en la interfaz, el framework de Avalonia con sus propiedades observables facilita la implementación del cliente.

4.2. Arquitecturas

La arquitectura del cliente ya se ha explicado por qué es MVVM, mientras que la del servidor es MVC, ya que permite la visibilidad de mensajes recibidos y emitidos por el servidor desde la consola (la vista del servidor) y la separación entre datos y entidades (el modelo del servidor) y la recepción y envío de mensajes (la conexión, el controlador del servidor).

4.3. Implementación del servidor

Se describen a mayor detalle las responsabilidades de los objetos mediante los diagramas del repositorio, pero con base en el diseño se implementa lo siguiente:

- **Entidades que contienen y validan datos:** forman parte del modelo y se implementan mediante los siguientes objetos.
 - Usuario del chat.
 - Sala del chat.
 - Reglas de texto
 - Excepciones:
 - Por nombre de sala fuera de rango.
 - Por nombre de usuario fuera de rango.
 - Por estado inválido del usuario: Cuando su estado se modifica a un valor no válido (como al mismo estado por ejemplo).
 - Por texto inválido: Cuando el formato del texto sale del rango de caracteres permitidos o tiene cualquier formato que no sea considerado en este modelo.
 - Por usuario ya existente: Cuando se intenta crear un usuario con un nombre que ya existe.
 - Por usuario no encontrado: Cuando se intenta encontrar a un usuario por algún identificador pero no se consigue.
 - Por sala ya existente: Cuando una sala ya existe y se intenta crear otra con el mismo nombre.
 - Por sala no encontrada: Cuando se intenta encontrar a una sala por algún identificador, pero no se consigue.
 - Por usuario no invitado: Cuando se intenta agregar un usuario a una sala sin haber sido previamente invitado.

- **Encargados de la conexión:**
 - Conexión con el cliente: permite enviar y leer mensajes de un cliente asociado a un usuario.
 - Manejador de la conexión con el cliente: maneja la conexión y desconexión segura de un cliente dado.
- **Notificaciones de entrada/salida de mensajes:**
 - Manejador de eventos de mensaje recibido.
 - Manejador de eventos de mensaje enviado.
- **Contenedor global de datos:**
 - Datos del chat: contiene y permite acceso y modificación a los usuarios y salas registrados.
- **Actores en el servidor:**
 - Interfaz de estrategia de acción respuesta: Define una acción a ejecutar para responder a un mensaje recibido por el servidor. Sus implementaciones son estrategias para cada uno de los doce tipos de mensajes que recibe el servidor, *i.e.* estrategias para: IDENTIFY, STATUS, USERS, TEXT, PUBLIC_TEXT, NEW_ROOM, INVITE, JOIN_ROOM, ROOM_USERS, ROOM_TEXT, LEAVE_ROOM, DISCONNECT.
- **Traductores de mensajes:**
 - Codificador UTF8: Traduce de bytes a string en formato UTF8 y viceversa. parámetros del mensaje.
 - Traductor de mensajes: Traduce un mensaje recibido por el servidor a una estrategia de acción respuesta.
 - Datos base de mensaje: define al menos un campo “type” para cada mensaje. Cada una de sus implementaciones es por cada uno de los doce tipos de mensajes recibidos por el servidor y contiene los campos necesarios para ello. Sirve para la lectura de mensajes.
 - Constructor de mensajes: Construye mensajes en formato JSON por medio de varias operaciones que definen los. Sirve para el envío de mensajes.

Un detalle no considerado en el diseño es el controlador y la vista. El controlador se encarga de iniciar el bucle de lectura y ejecución de estrategias del servidor, mientras que la vista se encarga de leer si el controlador indica que ha recibido o enviado un mensaje a algún cliente, para mostrarlo en consola. Sus implementaciones son dos clases: Controlador del servidor y Vista de mensajes, respectivamente.

4.4. Implementación del cliente

■ Entidades observadas:

- Usuario del chat: contiene los datos de un usuario enlazado con el cliente que usa la aplicación.
- Sala del chat: contiene los datos de una sala creada por el cliente o por otro usuario.
- Sala principal del chat: contiene los datos de la sala principal, información de registros de usuarios, etc.
- Mensaje de chat: enlaza el nombre de un emisor de un mensaje (que puede ser una sala o un usuario) con un texto (el contenido del mensaje).
- Invitación a sala del chat: contiene los datos de una invitación al cliente a una sala.

■ Conexiones con el servidor:

- Conexión del cliente: permite enviar y leer mensajes del servidor y se asocia al único usuario que el programa del cliente tiene responsabilidad de manejar su conexión (el propio cliente).

■ Emisores y receptores de mensajes:

- Receptor de mensajes: Sirve como interfaz para recibir mensajes del servidor a través de la conexión.
- Emisor de mensajes: Sirve como interfaz para enviar mensajes al servidor a través de la conexión, por cada tipo de mensaje que el cliente puede enviar.

- **Traductores de mensajes:** La conexión del cliente sirve como traductor de mensajes de bytes a cadenas, y el emisor y receptor como traductores de mensajes a acciones y viceversa, pero además están:

- Datos de mensaje: Contiene los campos necesarios para la serialización y deserialización de mensajes entrantes, con todos los parámetros que puede tener un mensaje en formato JSON recibido por el cliente y descrito por el protocolo.

■ **Vistas:**

- Vista de conexión: lo que se muestra para establecer la conexión con un servidor (admitiendo IP y puerto).
- Vista de registro: lo que se muestra para identificar al cliente en el servidor al que se conectó.
- Vista de chat: lo que muestra las vistas necesarias para brindar las funciones principales de la aplicación (envío de mensajes, visualización de lista de usuarios, etc.) al usuario desde la interfaz gráfica.
 - Vista de panel principal: muestra las opciones para invitar, crear sala, desconectarse y cambiar el estado, así como el nombre con el cual el cliente se registró.
 - Vista de panel de invitaciones: muestra las invitaciones a salas que el cliente ha recibido y permite aceptarlas/rechazarlas.
 - Vista de panel de usuarios: muestra la lista de usuarios de la sala (o chat privado) en la que el cliente escribe, así como sus estados. También permite al usuario seleccionar un usuario para entrar a su chat privado.
 - Vista del panel de salas: muestra todas las salas (incluyendo la principal) de las cuales el cliente es miembro, y permite seleccionar alguna para entrar en su chat.
 - Vista de panel de chat: muestra los mensajes de la sala (o chat privado con algún usuario) actual, así como un panel que permite escribir un mensaje y enviarlo a la misma.
 - Vista de panel de invitar usuario: muestra un panel que permite invitar a un usuario a una sala indicando ambos nombres.
 - Vista de panel de crear sala: muestra un panel que permite crear una nueva sala ingresando su nombre.

■ **Proveedores de contexto:**

- Un proveedor de contexto por cada vista descrita anteriormente.
- Vista-modelo de usuario de chat: Envuelve los datos de un usuario del chat para hacerlos observables y añadirle propiedades adicionales (como color al status).

- Vista-modelo de sala de chat: Cumple la misma función que la VM de usuario de chat con un usuario de chat, pero para la sala de chat.
- Vista-modelo de opción de estado: Tiene la información observable de un estado que puede ser seleccionado por el usuario, así como métodos para seleccionar las opciones válidas.

El proyecto no cuenta con la implementación completa del cliente que se tenía diseñada, faltan las vistas y proveedores de contexto relacionados con la creación, uso e invitación de salas, así como manejar eventos como la desconexión del cliente.

Capítulo 5

Conclusiones

El proyecto, aunque cuenta con la implementación completa del servidor, no cuenta con la implementación completa del cliente. Los principales retos y retrasos que culminó en este resultado fueron:

- La etapa de análisis y diseño se alargó más de lo necesario, durando hasta dos semanas antes de comenzar a escribir el servidor o cliente.
- El cliente se realizó no paralelamente al servidor, sino posterior a la implementación de este, tanto su diseño como implementación.
- Concluir del diseño que la implementación de una interfaz gráfica era necesaria retrasó el avance del cliente, dado que se introdujo un framework totalmente desconocido.
- La falta de experiencia con los patrones de diseño MVC y MVVM, con los patrones de diseño implementados (Como Builder, Strategy, etc.), con la construcción de interfaces gráficas, con el uso de sockets y con la programación asíncrona formaron un cuello de botella que concluyó en un servidor completo según el protocolo, pero en un cliente apresurado y no terminado.

Referencias

- [1] Ask Ubuntu Community. *How to install doxygen on ubuntu*. 2020. URL: <https://askubuntu.com/questions/1291874/how-to-install-doxygen-on-ubuntu>.
- [2] Avalonia UI Team. *Get Started — Avalonia Docs*. 2026. URL: <https://docs.avaloniaui.net/docs/get-started/>.
- [3] Lorenzo Carbonell. *Automatizar Git con Bash*. 2019. URL: <https://atareao.es/tutorial/scripts-en-bash/automatizar-git-con-bash/>.
- [4] Devgex. *Artículo Técnico 00043308*. 2026. URL: <https://devgex.com/es/article/00043308>.
- [5] Luis Llamas. *CLI de .NET: crear, compilar y ejecutar proyectos y ejemplos*. 2025. URL: <https://www.luisllamas.es/como-usar-cli-dotnet/>.
- [6] Luis Llamas. *Compilar C# desde consola, sin Visual Studio*. 2013. URL: <https://www.luisllamas.es/compilar-c-desde-consola-sin-visual-studio/>.